2024 API Security Market Report

APISEC UNIVERSITY

apisecuniversity.com

APISEC UNIVERSITY

2024 API Security Market Report Highlights

90% of API breaches result from a combination of Authentication, Authorization, and Data Exposure Flaws.

78% of respondents prefer "shift left" API security strategies over "shield right."

The "API security skills gap" remains the top concern for application security departments.

Organizations still rely on manual, infrequent, incomplete security testing of APIs, leaving most releases untested before production.

State of API Security

If "software is eating the world," then APIs are eating the Internet. In 2019, Akamai analyzed its network traffic and found API calls accounted for 83% of all traffic. That figure is likely over 90% now. But while APIs offer incredible benefits in simplifying coding, integration, and deployment, they have become, according to Gartner, the "most frequent attack vector."

Organizations of every size in virtually every industry rely on APIs to fuel innovation and digital transformation. APIs provide the "glue" that connects web UIs and mobile applications to backend data, transaction engines, and internal microservices. Attackers have discovered APIs make ideal targets for breach as they offer direct access to internal resources, are often over-permissioned, return more data than required, expose functionality unavailable in UIs, and more.

This report analyzes survey results from the 75,000 members of the APIsec University community, organized into three sections:

1. API Security Breaches Analysis
2. The 3 Pillars of API Security Framework.
3. Keeping APIs Secure

While API breaches are frequent and highly damaging, often resulting in the loss of 10s of millions of records, this report identifies concrete actions organizations can take to secure APIs and prevent breaches.

API Security Breaches

As part of the APIsec University curriculum, we regularly analyze every API-related security breach to understand its anatomy, root cause, and resulting impact. By learning about these breaches, security and development teams can better equip themselves to harden their APIs against similar vulnerabilities.

The following table samples the high-profile breaches over the past five years. What is striking is the sheer volume of successful breaches and the quantity of records stolen or lost to vulnerable APIs. Further, even organizations with extensive, sophisticated security teams weren't immune from breaches—evidence that API attacks can evade traditional application security technologies and techniques.

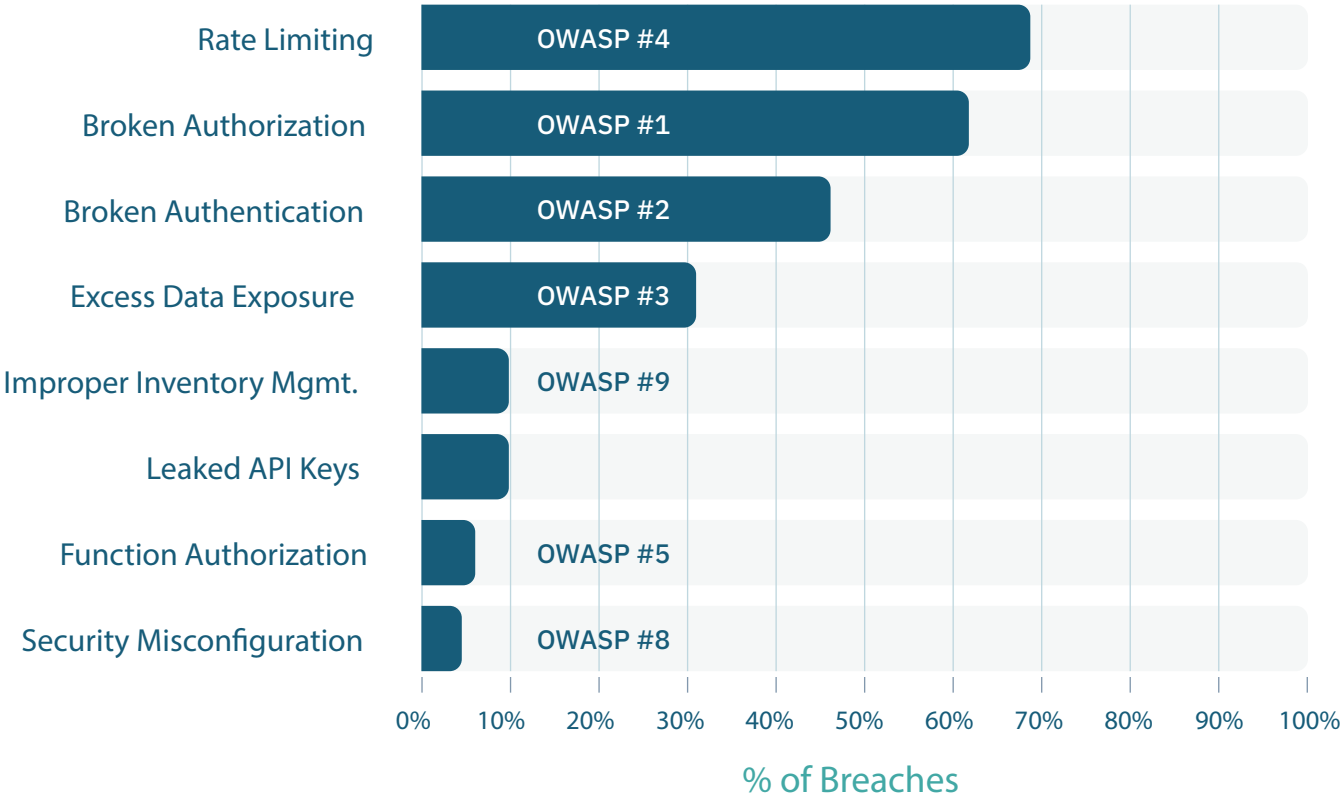
Target	Impact
Trello	15M
Peloton	2.6M records
Duolingo	2.6M records
Coinbase	Fraudulent transactions
USPS	60M records
Venmo	207M records
Experian	10s of millions
Instagram	Account takeover
Optus	10M records
Bumble	95M records
T-Mobile	30M records

Target	Impact
Facebook	530M records
John Deere	Account harvesting
Twitter	5.4M records
Sumo Logic	Key leak
Pokemon Go	Data exposure
Yandex	Hacktivism
Zoom	Unauthorized access
Linkedin	700M
Dropbox	Key leak
Tesla Backup	Data exposure
First American	885M

Target	Impact
7-Eleven 7Pay	Account takeover
Tinder	Data exposure
Waze	Data exposure
Wordle	Data manipulation
Echelon	Data exposure
Clubhouse	1.3M
Parler	70TB data harvested
Grinder	Account takeover
Ring App	Data exposure
Plenty of Fish	Data exposure
JustDial	100M

Breach Analysis

For this report, we analyzed over 50 publicly documented API breaches to identify the attack vulnerabilities exploited. The analysis reveals a high level of overlap in the attack vectors. Over 90% of the breaches studied resulted from just three API vulnerabilities (or a combination thereof).



Rate Limiting

With almost 70% of analyzed breaches revealing rate-limiting abuse, it is tempting to conclude that weak or missing rate limiting is the #1 issue to address to protect organizations from API breaches. Many breaches resulted in the loss of tens or even hundreds of millions of records, indicating a failure to detect and block high-volume attacks.

Nevertheless, rate-limiting abuse is not the core cause of API breaches. If there is nothing valuable to harvest, attackers would not be motivated to perform high-volume actions. Weak or missing rate limiting worsens breaches but is not the root cause. More often than not, the core issue is APIs that reveal valuable data—sensitive information that can be stolen and sold or used maliciously.

The prevalence of rate-limiting abuse also reveals the limitations of runtime defenses. Most larger organizations have sophisticated cybersecurity teams and employ technologies to detect and block high-volume attacks, such as web application firewalls (WAF). In this arms race, attackers use techniques to evade detection, including rotating IP addresses, using proxies, and other methods.

Authorization, Authentication and Data Exposure

Moving beyond rate-limiting, the analysis identifies three vulnerabilities present in over 90% of the breaches. These three, not coincidentally, map to the top three items on the OWASP API Security Top 10:

OWASP API 1: Broken Authorization

Broken Authorization, also known as Broken Object Level Authorization in the OWASP API Security Top 10, refers to application logic that allows manipulation of object IDs/references to access data that a user should not be able to access. Attackers most commonly exploit this to access and harvest records belonging to other users on the platform. Incremental user IDs can make data harvesting much easier for attackers as they can iterate through millions of records.

OWASP API 2: Broken Authentication

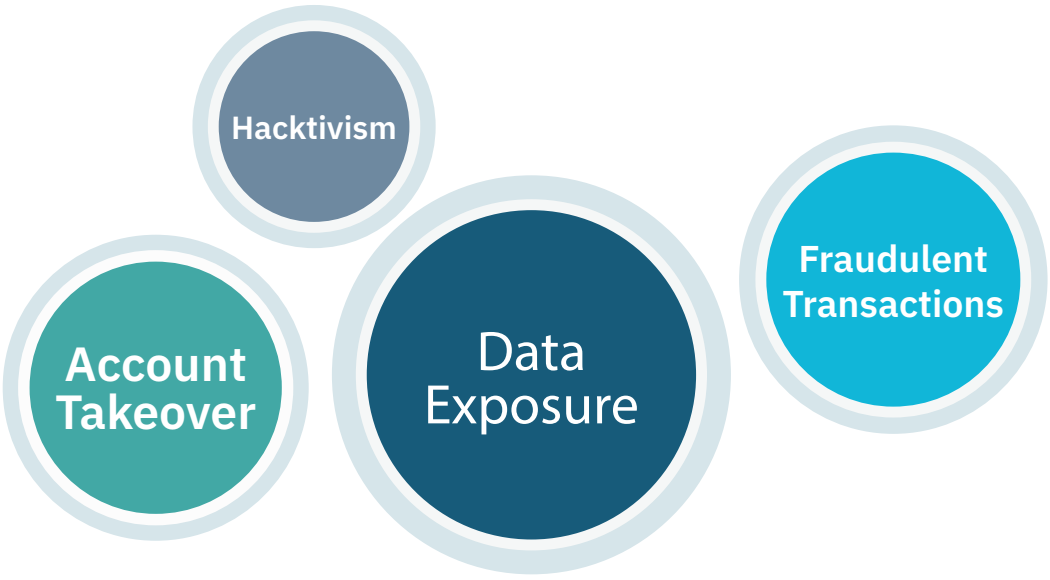
Broken Authentication refers to APIs with weak or poorly implemented authentication mechanisms, which may result from weak password requirements, vulnerability to password stuffing attacks, lack of CAPTCHA, and poor password reset processes. However, the vast majority of breaches with "broken authentication" resulted from a complete lack of Authentication, i.e., the APIs were left open with no password requirements or other Authentication.

OWASP API 3: Excess Data Exposure

Excess Data Exposure, an element of OWASP API #3, Broken Object Property Level Authorization, reflects a vulnerability where APIs return unnecessary, sensitive information. API endpoints that return more data than necessary and then rely upon the user interface to filter or limit what a user can see are frequent contributors to excess data exposure.

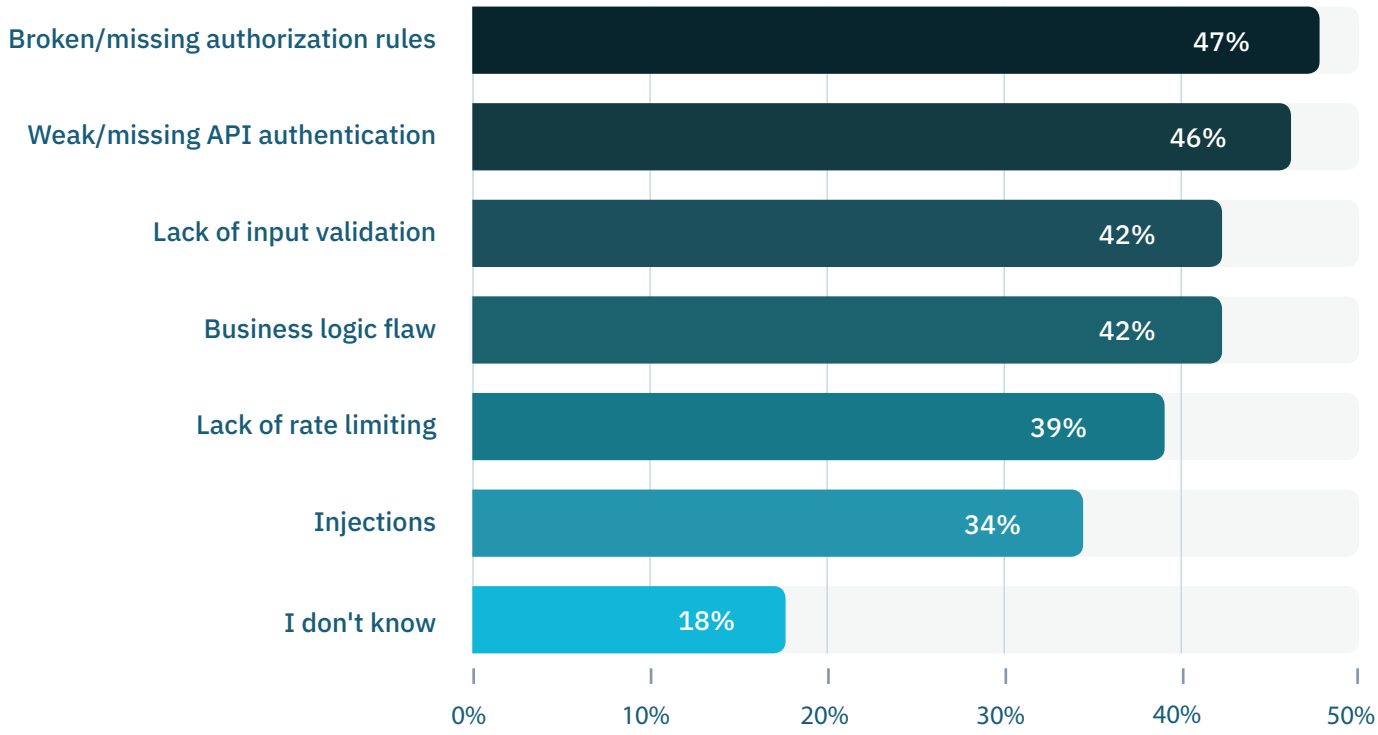
The breach analysis reveals that organizations can dramatically increase API security by addressing these three vulnerabilities first. The API Lifecycle section of this report guides how best to detect and remediate these vulnerabilities.

Which types of API risks are you most concerned about?



Data exposure leads the list of concerns for survey respondents, with over 80% indicating this is a top issue. Account takeover and fraudulent transactions also appear prominently, with over 50% of respondents for each. Respondents also shared concerns about business logic flaws, lost API keys, and authorization gaps.

Which vulnerabilities have you experienced? (Select all that apply)

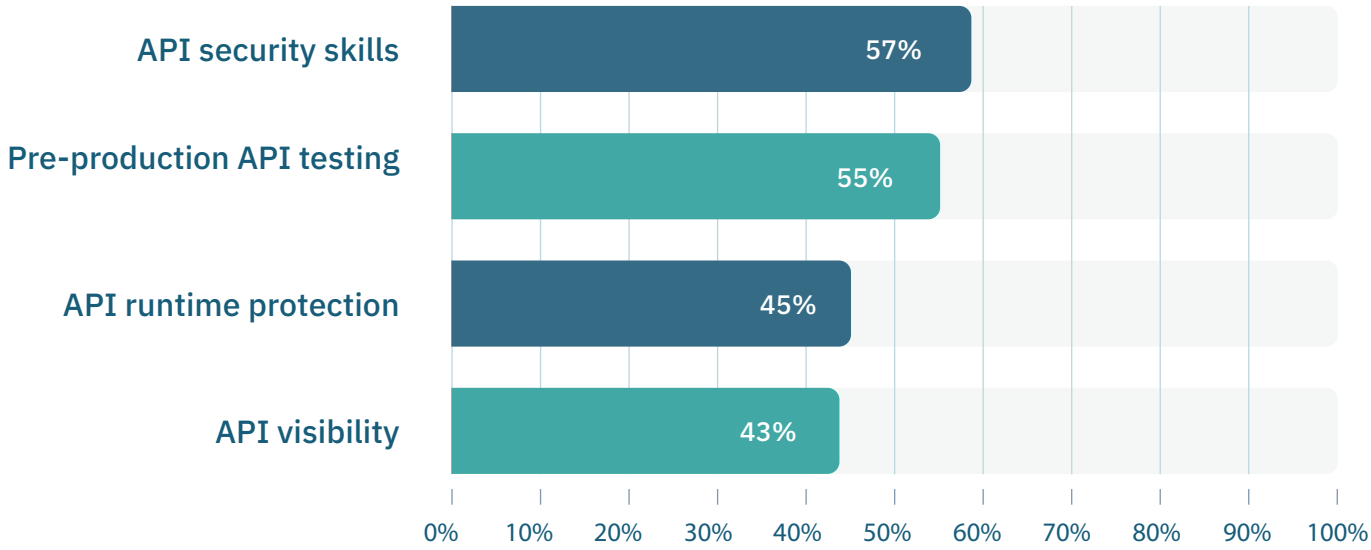


Survey responses aligned very closely with the OWASP API Security Top 10, with Broken Authorization (OWASP API #1) and Broken Authentication (OWASP API #2) ranking as the top two most common vulnerabilities.

RECOMMENDATION:

As highlighted in the Breach Analysis, addressing the highest priority OWASP API Security Top 10 vulnerabilities offers the most significant improvement in overall security posture. Organizations should frequently test for Authentication, Authorization, and Data Exposure vulnerabilities.

What is your biggest API security challenge?



Not surprisingly, the number one challenge reported by survey respondents is a need for API security skills. This challenge falls in line with the general shortage of cybersecurity professionals. It is, perhaps, even more pressing with API security as this area is new, and even fewer experts exist despite being recognized by Gartner as the "most frequent attack vector."

API security testing follows a close second to the skills gap and is well ahead of runtime API protection and API visibility. While all are highly important for a complete API security program, API security testing has emerged as a significant gap in the layers of cyber defense.

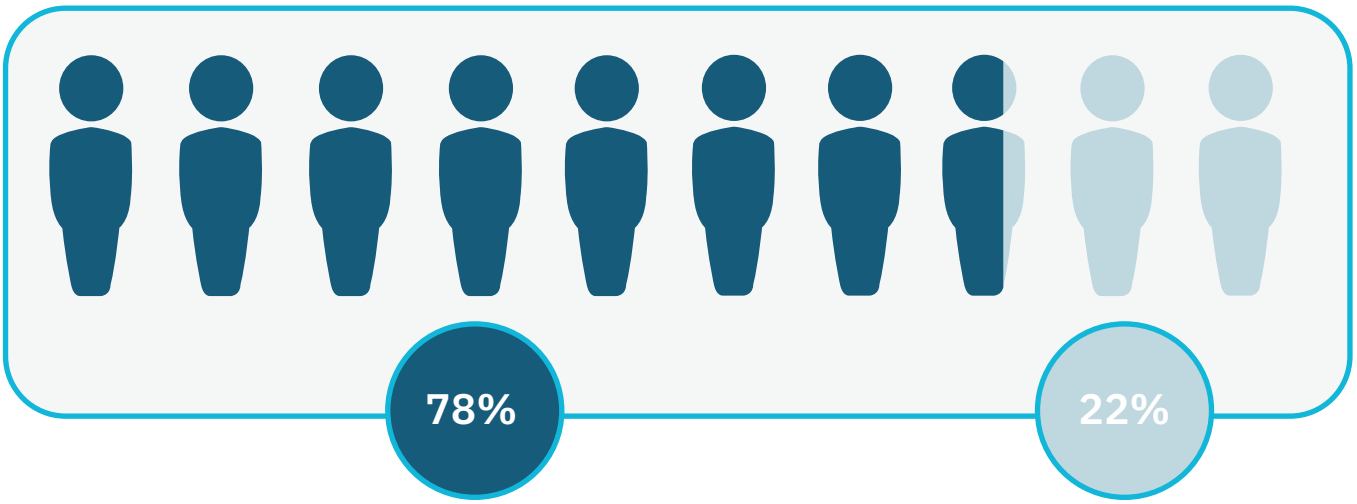
RECOMMENDATION:

Invest in upskilling the team you have. Organizations should ensure that not only security teams understand API threats but engineering, operations, and other groups do as well. By raising awareness of API risks and best practices, your API developers and product managers can create more resilient, secure APIs during the coding phase.

PRO TIP:

Schedule an API Security Workshop for your organization. These 1-hour educational sessions are free and delivered by APIsec University instructors.

What is your preferred strategy for securing APIs?



Shift Left:
Discover vulnerabilities before production with testing

Shield Right:
Automate detection and blocking of APIs in runtime

In the API security community, there is significant debate, particularly among vendors, about "shift left" security approaches vs. "shield right." Generally, shift left refers to incorporating security into the pre-production software lifecycle via automated testing, typically in the CI/CD pipeline. Shield right approaches rely on the real-time threat detection and protection capabilities of a live API in production.

An overwhelming 77% of respondents preferred to shift left, recognizing the benefit of discovering vulnerabilities earlier than later, preferably before production. This preference grows to over 80% for organizations with over 10,000 employees.

RECOMMENDATION:

As discussed later in The API Lifecycle, continuous, automated, pre-production testing of APIs is one area where many organizations can significantly improve. Organizations should evaluate technologies that can operate within the SDLC.

The 3 Pillars of API Security

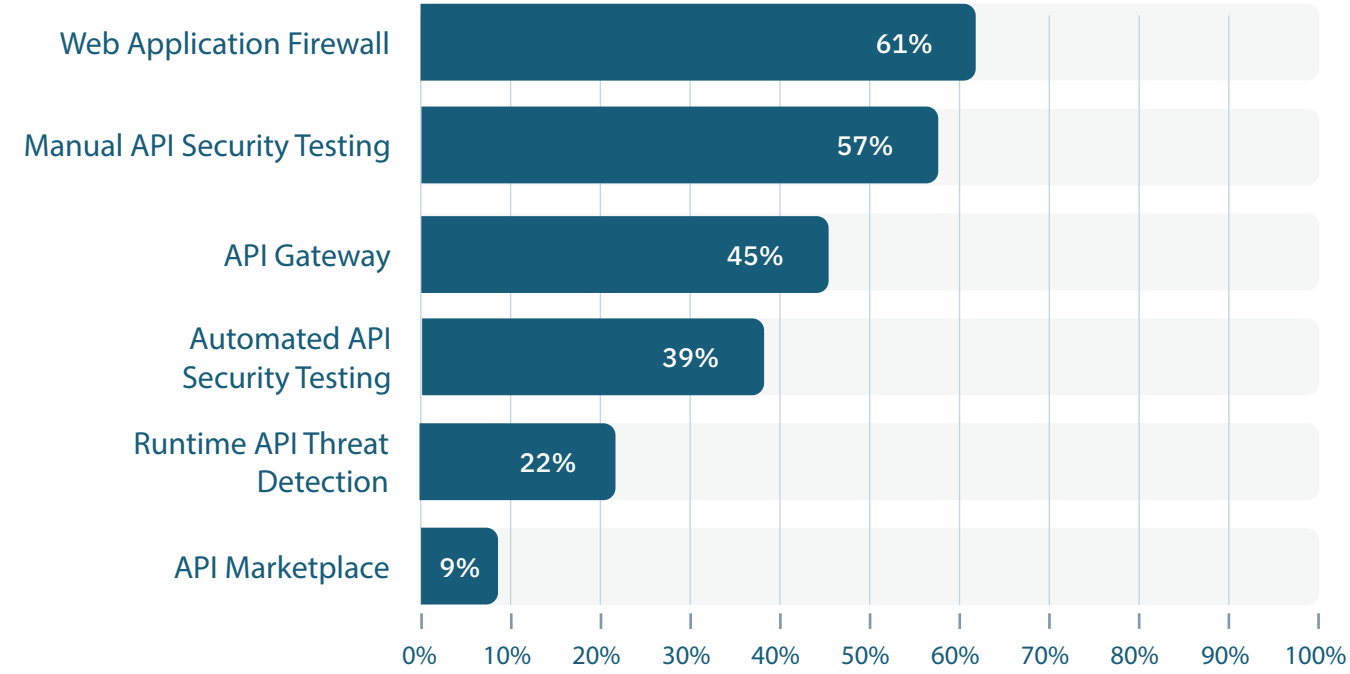
The concept of the 3 Pillars of API Security is present across all of the courses on APIsec University. This simple framework effectively covers the foundational requirements for a strong API security program.

Governance covers how an organization defines, develops, and deploys APIs. Governance typically has two major components: 1) API visibility and awareness and 2) policy and process. Organizations must have visibility into all APIs operating/used in their environment, including details such as how they are secured and what data they can access. In addition, organizations should thoughtfully define their API development policies and processes, documenting the API creation, testing, deployment, and retirement phases.

Testing ensures that APIs are evaluated against a suite of attack types to identify vulnerabilities and flaws. Manual API testing cannot keep up with the rapid pace of engineering teams that often push new code into production monthly, weekly, or even daily. API testing must be automated, comprehensive (covering all API endpoints across all OWASP categories and more), and continuous (integrated into CI/CD pipelines to test every release)..

Monitoring analyzes APIs' real-time behavior, looking at live traffic and applying policies, threat detection, and blocking capabilities. Gateways and WAFs are the most common monitoring technologies. Gateways are API management tools that incorporate some security features such as user authentication, traffic filtering, and input validation. WAFs can be used for rate limiting, threat detection, and API discovery.

Which of the following do you use in your organization?



This survey question examines the multiple layers of API security across the lifecycle of APIs. As expected, Web Application Firewalls (WAF) ranked highest - this technology has existed for over a decade and is widely used to protect applications from threats.

Manual API testing ranked second, significantly ahead of automated API testing, reflecting a continued reliance on human testing. The infrequency of security testing starkly contrasts with functional/unit testing (largely automated by QA teams), where application updates are consistently tested for functionality ahead of production. Manual testing is time and labor-intensive, so APIs are rarely security-tested to the breadth and depth needed before production.

API Gateways are widely used, especially among organizations with over 1,000 employees. Many larger companies reported using multiple gateways from multiple vendors to manage high volumes of APIs.

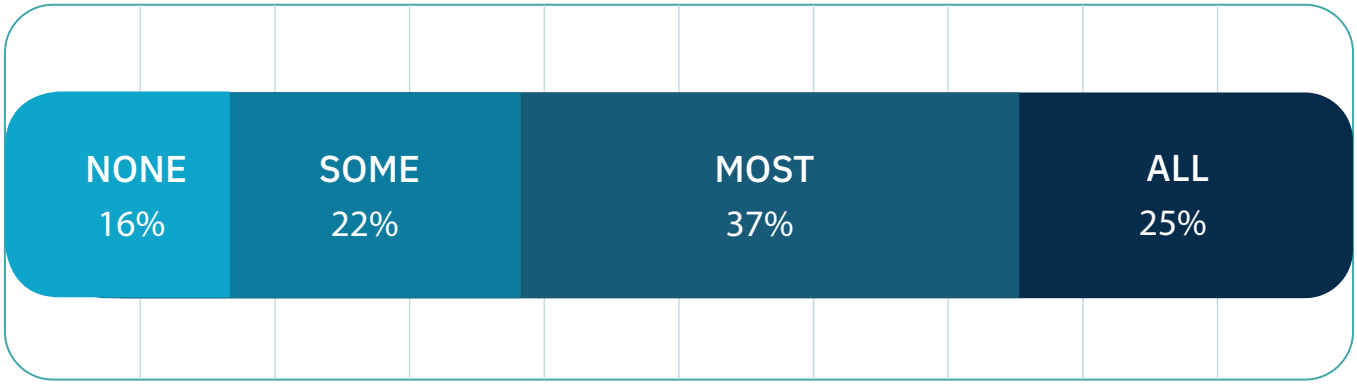
Runtime API threat detection technologies are a last line of defense that watch live API traffic and typically use baselines, anomaly detection, and other algorithms to identify attacks at runtime. Respondents reported only 22% adoption of these runtime threat detection solutions. WAFs are increasingly incorporating monitoring solutions as a natural extension as they already sit in line, have visibility to API traffic, and can block traffic as needed.

RECOMMENDATION:

API consistency is foundational in the 3 Pillars of API Security. Organizations should consistently develop, document, test, and operate their APIs in the same manner - ensuring their entire API inventory looks, behaves, and runs the same way. API Gateways provide a management layer that provides DevOps teams with a platform to enforce consistency. Organizations should also manage all APIs in a centralized gateway with standard policies and enforcement.

Speak with your WAF vendor to understand their API-specific security capabilities. Many now incorporate API discovery capabilities, API traffic anomaly detection, and other security features.

How complete is your API inventory?



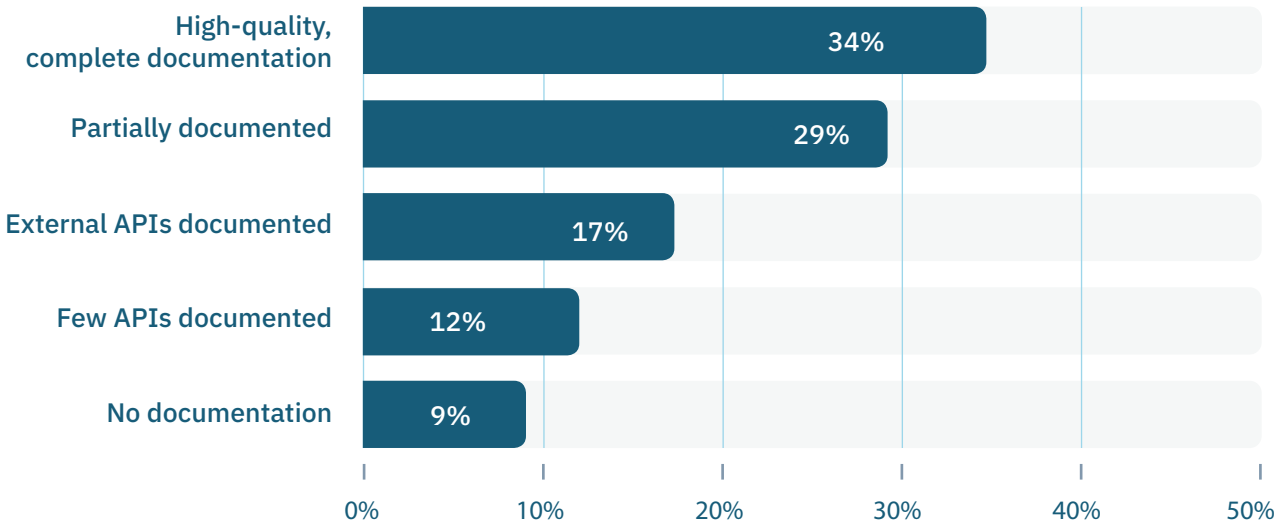
- We don't have an API inventory currently
- We know some, but many are still unknown
- We know most - but there probably are some hidden ones
- We know all our APIs

Roughly 2/3 of respondents say they know all or most of their APIs. Clearly, there is significant room for improvement. Lack of complete API visibility is one of the most common complaints of security teams.

RECOMMENDATION:

Increase collaboration between security and engineering departments. While APIs may be unknown to security teams, this is rarely true among engineering teams. Many teams rush to purchase technologies that sit on the network, watch traffic, and identify APIs that cross the wire to "discover" APIs. This is one approach to discovering APIs; a more straightforward starting point is engaging with the engineering team to map out APIs. An API Marketplace or Gateway can provide a repository for developers to post their API creations. Increasing communication between developers and security teams is critical, not just for visibility, but to improve security during coding.

What best describes your API documentation?



After identifying APIs, the next concern is documentation—are these APIs adequately documented? API documentation is the foundation for strong governance, effective API security, and achieving API business goals. The survey results show a general need for more consistency in API documentation.

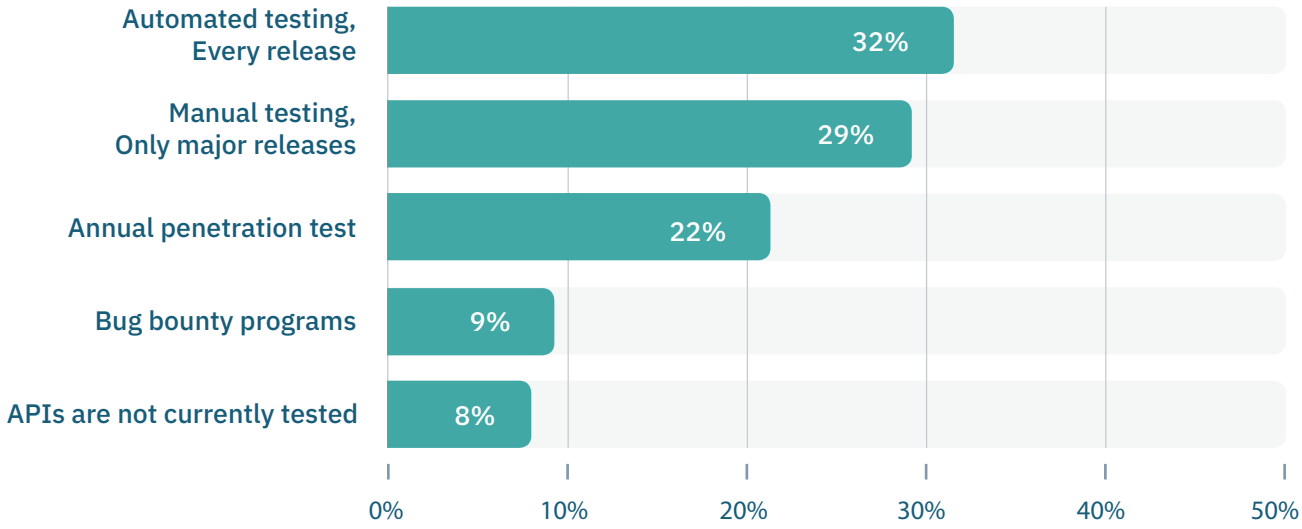
- 66% have incomplete or missing documentation of their APIs
- Only 34% have high-quality documentation of both internal and external APIs

RECOMMENDATION:

APIs are reusable resources that can be leveraged by other consumers (internal or external) to access information and functionality and enable simple integrations easily. Yet reusability requires clear API documentation that informs security teams (and technologies) what the API does, its capabilities, what data it accesses and exposes, and how it is secured. Documentation must be mandatory for all APIs.

PRO TIP:
Check out the API Documentation Best Practices course on APIsec University taught by Jason Harmon, the leading expert on documentation.

What best describes your API security testing approach?



We see a significant gap in most organizations' approaches to API security testing. It's typical for new code to be pushed to production every month/week/day. But API security testing is still largely manual, often performed once or twice a year by pen-test teams or not at all. There's a huge mismatch in the pace of development and frequency of testing, causing releases to go live without thorough security testing.

RECOMMENDATION:

Match testing to the pace of delivery to test APIs on every release against a comprehensive suite of attack scenarios (all of the OWASP API Top 10 at minimum) as part of the CI/CD pipeline. Ensure testing keeps pace with releases and no APIs go live without proper vetting.

PRO TIP:

Test your API immediately with APIsec Scan, a free utility created by APIsec University. This scan will reveal unsecured API endpoints, high-risk endpoints, and other issues.

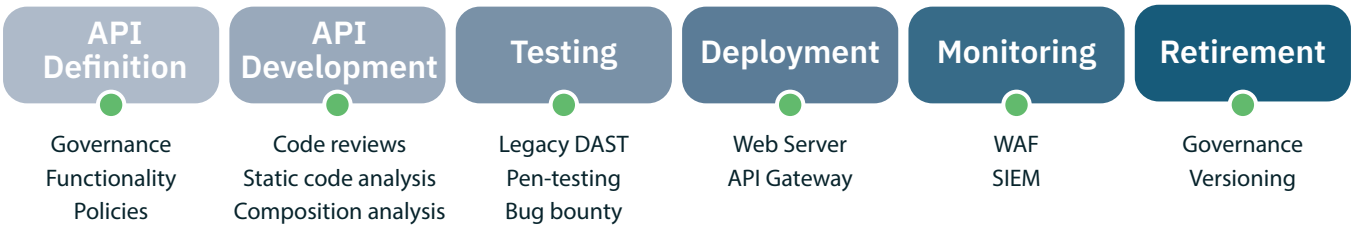
Keeping APIs Secure

The survey data and analysis make clear that APIs are, as stated by Gartner, "the most frequent attack vector" and have resulted in a constant stream of breach announcements. Given APIs' power to rapidly access data and the ability for bad actors to script API attacks, breaches often result in millions, or hundreds of millions, of records stolen.

What can organizations do to build more secure, resilient APIs and protect data from exposure in light of this situation? The breach analysis identified three critical vulnerabilities that combined account for over 90% of all breaches: broken Authorization, missing Authentication, and Excess Data Exposure. Next, this report examines the API lifecycle and where these vulnerabilities can best be uncovered and fixed.

The API Lifecycle

The entire API lifecycle must integrate security, from initial ideation to development, deployment, and retirement. The graphic here provides a simplified view of the API Security Lifecycle, featuring the significant phases from when APIs are first contemplated to when they are retired.



At each stage of the lifecycle, organizations can enhance security and achieve defense-in-depth.

API Definition: Here, organizations can leverage their API security policies to ensure security is considered in the first discussions. Key considerations include what function this API serves, what data it will access/expose, how access should be controlled, where it will operate, and what testing requirements should be required.

API Development: Code reviews, static code analysis, and software composition analysis are fundamental best practices for identifying common coding flaws, vulnerable libraries, and other issues. While this capability is a must, none of the API breaches listed in this report resulted from an injection attack or vulnerable library.

Testing: Traditional dynamic application security testing (DAST) tools have focused primarily on web and mobile applications, testing at the UI layer. These technologies tend to find common vulnerabilities, such as SQL injections and cross-site scripting but have limited API-specific capability. Organizations are left with manual API testing (pen-testing, bug bounties, red teams).

Deployment: APIs are typically deployed, operated, and managed in API Gateways. While primarily a management platform, API gateways also offer some security capabilities. These include user authentication, high-level authorization controls, and input validation rules.

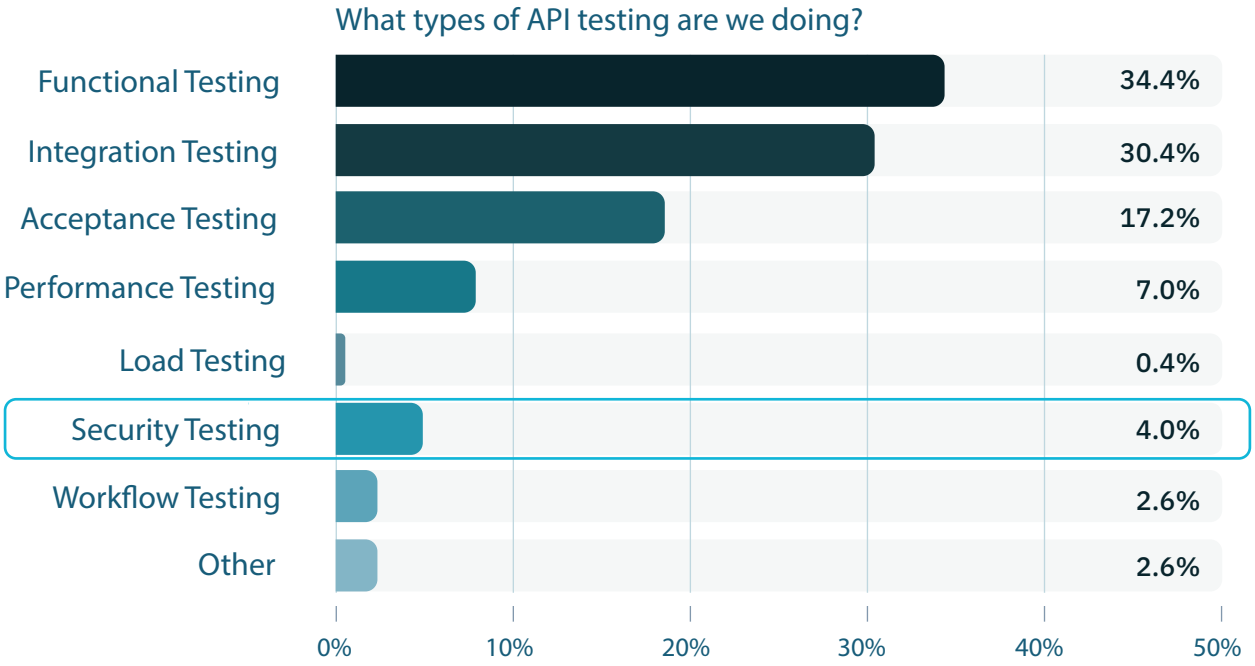
Monitoring: Frequently, organizations rely on web application firewalls to monitor live traffic on APIs for detectable threats and actively block attacks. New API threat detection solutions offer API-specific capabilities.

Retirement: Finally, organizations must be as considerate about retiring APIs as launching them. Attackers look specifically for out-of-date, unretired versions of APIs as these frequently have lower levels of security and may include un-deprecated endpoints and other flaws.

The API Security Gap

Almost 80% of respondents answered shift left when asked about their preferred API security strategy. The reasoning is quite apparent: finding vulnerabilities earlier in the SDLC is less expensive and easier to remedy and ensures vulnerabilities are discovered and fixed before production.

However, there is a clear API security gap between Development and Deployment. Testing remains incomplete, highly manual, and point-in-time. Further, according to the State of APIs report from Rapid in 2022, security testing accounts for only 4% of all API testing efforts. It is safe to say no code goes into production without thorough functional testing - however, the same is far from true for security testing.



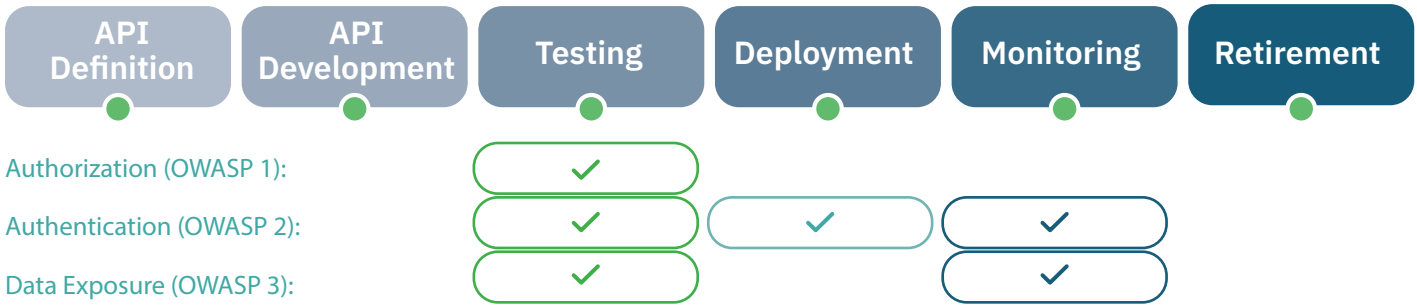
Source: www.stateofapis.com

The API security testing gap is made worse by several important factors:

1. APIs are machine interfaces, not human interfaces. Their nature makes manual testing extraordinarily time-consuming and challenging to master.
2. While engineering teams push new code to production every month, week, and day, manual testing cannot keep up, leaving most releases untested.
3. A typical API has 50-100 different endpoints; each needs testing for Authorization, Authentication, data exposure, fuzzing, injections, and many more attack vectors. Comprehensive testing frequently requires over one thousand attack scenarios.

Addressing the API Security Gap

Having identified the most significant contributors to API breaches - authentication, authorization, and excess data exposure - the next challenge is to evaluate how best to address these weaknesses. To this end it is helpful to revisit the API Security Lifecycle, this time with an eye to where in the lifecycle vulnerabilities can be found.



Having identified the most significant contributors to API breaches—Authentication, Authorization, and Excess Data Exposure—the next challenge is to evaluate how best to address these weaknesses. To this end, it is helpful to revisit the API Security Lifecycle, this time with an eye on the steps in the lifecycle where vulnerabilities can be found.

While some flaws, such as broken Authentication, can be found during testing, deployment, and monitoring, security can only reliably discover others in the testing phase. Monitoring approaches rely on complex algorithms, traffic baselining, statistical analysis, and anomaly detection to identify unusual and potentially malicious traffic. However, security can test for all of these API flaws before production. For example, identifying Authorization flaws such as the ability to access another's account can be revealed by running a logic test where User A's account attempts to access User B's.

Similarly, security can automate tests to query every API endpoint without credentials to determine if any respond, which is evidence of unsecured endpoints. Security can uncover Excess data exposure by testing every endpoint and examining what data it delivers - is there PII, sensitive business data, or anything else that is not required by the use case?

Further, as evidenced by the shift left vs shield right response, finding API flaws earlier in the lifecycle (particularly before production) is highly preferred. Testing can and should be automated and integrated into software release pipelines to ensure security vulnerability testing for every API before moving into production.